

The Attack Category Gap: How Constrained-Header Design Determines Routing Attack Taxonomy in LoRa Mesh Protocols

August Chao

Abstract—Formal security models for mesh routing protocols predict that an attacker who controls routing state can become an adversarial relay—intercepting, inspecting, or dropping traffic. We show that this prediction, while formally correct, masks a categorical difference in deployed behavior that determines the appropriate defense architecture. In LoRaMesher, a field-collapse design (`dst = next_hop`) causes poisoned packets to reach an absent destination, producing a *silent black hole*: the gateway observes zero PDR, but the attacker never receives payload. In Meshtastic v2.6+, separate `to` and `next_hop` fields allow the attacker to become an *active relay*: the gateway observes 100% PDR while the attacker intercepts every packet. We validate both behaviors on physical EoRa PI hardware (6× ESP32-S3 + SX1262 boards). We further show that the attack category difference implies a defense category difference: relay-side anomaly detection closes the LoRaMesher gap (FNR $\approx 0\%$) but is fundamentally blind to the Meshtastic attack (FNR = 100% for all passive IDS strategies), and HMAC on control messages closes LoRaMesher but not Meshtastic. The correct Meshtastic fix binds `next_hop` in AEAD additional authenticated data—a property we formally verify with Tamarin Prover (L3 No_Relay_Takeover: proved after patch in 22 s). Our results yield concrete design guidelines for constrained-airtime mesh protocols: header field separation creates active-relay exposure, and any unprotected routing hint field requires cryptographic binding to the payload ciphertext.

Index Terms—LoRa mesh security, formal verification, Tamarin Prover, routing attack taxonomy, active relay, silent black hole, Meshtastic, LoRaMesher, intrusion detection, constrained protocols

I. INTRODUCTION

LoRa mesh networks are increasingly deployed in settings where network availability and data confidentiality are operationally critical: precision agriculture, disaster-response relay, and environmental monitoring. Two open-source firmware stacks dominate this space: LoRaMesher, a distance-vector routing protocol for ESP32 nodes, and Meshtastic, a managed-flooding protocol now running on several million devices globally. Both operate under extreme constraints—255-byte LoRa payloads, 1% ETSI duty-cycle limits, and software-only cryptography on microcontrollers.

Formal methods applied to routing security typically model the attacker as a Dolev-Yao network adversary who can intercept, replay, and inject routing control messages. At this abstraction level, both protocols are predicted to be vulnerable to adversarial relay takeover. Yet we observe a striking

discrepancy when we execute the formally predicted attacks on physical hardware: the two protocols produce categorically different outcomes under what should be the same attack.

The central finding of this paper is that this discrepancy is not a modeling error. It is a direct consequence of a single header semantic design choice: whether the routing hint field (`next_hop`) is represented as a *destination override* or as an *advisory relay field*. This choice determines (1) which attack category the protocol is susceptible to, (2) which IDS architecture can detect the attack, and (3) at which protocol layer the defense must be applied.

A. Contributions

- C1** First formal characterization of the formal-firmware divergence at the header-semantics layer in LoRa mesh protocols, using Tamarin Prover to show both protocols share the same formal vulnerability (L2, L3 FALSIFIED) while exhibiting categorically different hardware behavior (§III).
- C2** Cross-protocol attack taxonomy: same formal prediction, different attack categories. LoRaMesher yields a silent black hole (gateway PDR = 0%); Meshtastic yields an active relay (gateway PDR = 100%). Hardware validation on 3× EoRa PI boards, 120 s, 100% intercept rate (§V).
- C3** Passive IDS is fundamentally blind to Meshtastic active relay: FNR = 100% for all strategies including gateway-centric and relay-side destination-binding, established via E17 hardware experiment (§VI).
- C4** Patch asymmetry: HMAC applied at the same primitive level produces asymmetric outcomes across protocols (LoRaMesher: 7/7 attacks blocked; Meshtastic: 7/7 attacks succeed) because the attack vector and patched layer do not overlap (§VII).
- C5** Formal verification of the correct Meshtastic patch: binding `next_hop` in AEAD AAD proves L3 No_Relay_Takeover in 22 s under Tamarin 1.13.0, Docker image `lora-mesh-tamarin` (§VIII).

II. BACKGROUND

A. LoRaMesher: Distance-Vector Routing with Field Collapse

LoRaMesher implements a lightweight distance-vector routing protocol for ESP32 LoRa nodes. Each node maintains a routing table mapping destination addresses to (next-hop, metric) pairs. Routing control messages (Hello/RouteUpdate)

are broadcast periodically; nodes update their routing tables upon receiving better-metric routes.

Critical design choice: LoRaMesher encodes the next-hop address in the `dst` field of the LoRa packet header—the same field used to determine whether a node should receive the packet. Formally:

$$\text{dst}(pkt) = \text{next_hop_to}(\text{final_dst}(pkt)) \quad (1)$$

A node processes packet p only if $\text{dst}(p) = \text{self}$. This *field collapse* means that when an attacker poisons `next_hop` to an absent node, the packet reaches that absent node’s address—and no honest node processes it. The result is a silent black hole.

B. Meshtastic v2.6+: Managed Flooding with Separate Fields

Meshtastic uses managed flooding with an optional relay-hint optimization. Each `MeshPacket` carries two addressing fields:

- `to`: final destination (typically the gateway, `0xFFFFFFFF` for broadcast)
- `next_hop`: advisory relay hint—the node that should forward this packet

A relay node R processes a packet if `next_hop = R.id`. Crucially, R is reachable as a relay regardless of `to`. When an attacker sets `next_hop = attacker.id`, the packet is routed to the attacker, who receives and can inspect the payload before forwarding.

C. LoRa Physical Layer Constraints

Both protocols operate over LoRa (Long Range) radio, a chirp spread spectrum modulation. The constraints relevant to security design are:

- **255-byte maximum payload:** the SX1262/SX1276 LoRa PHY limits each packet to 255 bytes. Every header byte directly reduces usable data capacity.
- **1% ETSI duty-cycle limit:** in the 863–870 MHz and 915–928 MHz bands, nodes may transmit at most 1% of the time. At SF9/BW125 (airtime ≈ 2.4 s per 233-byte packet), a node can send at most one packet per 240 s on a single channel.
- **Broadcast channel:** LoRa uses a shared broadcast medium with no link-layer acknowledgements. Any node in range receives every transmission. This means any attacker in physical range can observe all traffic passively, making passive IDS strategies with promiscuous listeners feasible but also making all traffic observable to the attacker.
- **No hardware addressing:** unlike WiFi (which filters frames at the MAC layer by BSSID/SSID), LoRa delivers all packets to the receiver regardless of address. Packet filtering is done entirely in software by the receiving firmware—the `dst` or `next_hop` check is a software decision, not a hardware one. This is why field-collapse attacks route packets to absent nodes rather than causing PHY-layer discard.

These constraints explain why compact header designs (field collapse, minimal byte overhead) arose naturally in LoRaMesher. The confidentiality consequence of the separated-field design (Table I) was not a primary design consideration when routing table overhead dominated packet budgets.

D. Header Field Comparison

Table I summarizes the packet header structures of both protocols as deployed in firmware. The critical difference is in how the forwarding hint is represented relative to the receive predicate.

TABLE I
PACKET HEADER FIELD COMPARISON

Field role	LoRaMesher	Meshtastic v2.6+
Final destination	implicit (known at src)	<code>to</code> (explicit)
Next-hop / relay	<code>dst</code> (same field)	<code>next_hop</code> (separate)
Receive predicate	<code>dst == self</code>	<code>next_hop == self</code>
If field poisoned	<code>pkt reaches absent node</code>	<code>pkt reaches attacker</code>
Attacker gets payload?	No	Yes
Gateway PDR under attack	0%	100%

The LoRaMesher header is 7 bytes total (`dst` 2B + `src` 2B + `type` 1B + `metric` 2B). The Meshtastic `MeshPacket` proto header is 12–16 bytes (from 4B + `to` 4B + `id` 4B + `hop_limit` 1B + `next_hop` 1B + `flags` 1B). Both operate within the LoRa 255-byte payload constraint, but Meshtastic requires approximately 5–9 additional header bytes to carry the separated addressing fields.

E. Threat Model

We model a Dolev-Yao adversary with LoRa physical-layer capabilities: the attacker can receive all transmissions in range, transmit arbitrary packets, and replay captured packets. The attacker cannot break HMAC-SHA256 or AES-128 (computational security assumption). The attacker is a single network-internal node (one compromised ESP32-S3 device).

We do not model multi-attacker coordination or physical-layer attacks (jamming, signal injection). The focus is the routing control plane.

III. FORMAL MODEL: BASELINE VULNERABILITY

A. Tamarin Encoding

We encode both protocols in the applied pi-calculus using Tamarin’s multiset rewriting formalism [1]. Each model consists of honest node rules (sender, relay, gateway), attacker rules, and security lemmas. Models are available at `phase2_tamarin_models/meshtastic/` in the artifact repository.

Listing 1 shows the four core rules of the Meshtastic model. The `Packet_Send` rule captures the sender using a `NeighborEntry` (populated by poisoned `NodeInfo`) to select relay. The `Relay_Forward` rule fires when `relay = self`—this is the receive predicate in Tamarin terms. The

Attacker_Hijack_NextHop rule replaces relay with 'attacker' in any observed packet.

Listing 1. Meshtastic Tamarin model: core rules (meshtastic_routing.spthy)

```
rule Packet_Send:
  [ !Node(src, 'honest'), !NeighborEntry(src, relay,
    seq_r),
    !ChanKey(src, psk), Fr(~pkt_id) ]
  --[ PacketSent(src, 'gateway', relay, ~pkt_id) ]->
  [ Out(<'MeshPacket', src, 'gateway', relay, ~pkt_id,
    senc('data', psk)>) ]

rule Relay_Forward:
  [ In(<'MeshPacket', src, dst, relay, pkt_id, enc>),
    !Node(relay, 'honest'), !ChanKey(relay, psk) ]
  --[ PacketRelayed(relay, src, dst, pkt_id) ]->
  [ Out(<'MeshPacket', src, dst, 'none', pkt_id, enc>) ]

rule Attacker_Hijack_NextHop:
  [ In(<'MeshPacket', src, dst, relay, pkt_id, enc>),
    !Node(src, 'honest') ]
  --[ NextHopHijacked(src, dst, pkt_id) ]->
  [ Out(<'MeshPacket', src, dst, 'attacker', pkt_id,
    enc>) ]

rule Attacker_Drop_Packet:
  [ In(<'MeshPacket', src, dst, 'attacker', pkt_id, enc>) ]
  --[ PacketDropped(src, dst, pkt_id) ]->
  [ ]
```

The LoRaMesher model differs in exactly one rule: Relay_Forward matches on dst instead of the relay slot:

Listing 2. LoRaMesher Tamarin model: relay rule (baseline_lora_dv.spthy)

```
rule Node_Forward:
  [ In(<src, self, metric, payload>), // dst=self
    (field collapse)
    !Node(self, 'honest') ]
  --[ PacketRelayed(self, src) ]->
  [ Out(<src, next_hop_to_gw(self), metric,
    payload>) ]
```

At the symbolic level, both relay rules fire when “the routing slot matches self.” The Tamarin solver cannot distinguish whether this slot is the final destination or a separate advisory field: in both models the counterexample to L3 places 'attacker' in the relay slot via Hijack_NextHop. The formal divergence is structurally identical; the deployed behavioral divergence is not.

The three attacker rules are shared across both models:

- **Forge_NodeInfo**: inject control packet with arbitrary src, populating victim's NeighborEntry with relay='attacker'
- **Hijack_NextHop**: replace relay slot in any observed MeshPacket
- **Drop_Packet**: discard packet when acting as relay

B. Lemma Definitions

We verify five lemmas per protocol:

- **L1 (NodeInfo_Authenticity)**: $\forall$ trace, if node B installs a route via A , then A sent the corresponding control message.
- **L2 (Packet_Reachability)**: $\forall$ trace, if S sends p to G , then G eventually receives p .

- **L3 (No_Relay_Takeover)**: $\forall$ trace, no packet originating from S is relayed by the attacker.
- **L4 (Admin_Authenticity)**: $\forall$ trace, admin commands are authenticated to the claimed sender.
- **L5 (Executability)**: $\exists$ trace where S sends p and G receives it (sanity check).

C. Results

TABLE II
TAMARIN VERIFICATION RESULTS: BASELINE MODELS

Lemma	Type	LM	Mesh	Steps
L1 Node-Info_Auth	all-traces	FALSFD	FALSFD	5
L2 Packet_Reach	all-traces	FALSFD	FALSFD	6
L3	all-traces	FALSFD	FALSFD	4
No_Relay_TO				
L4 Admin_Auth	all-traces	FALSFD	FALSFD	9
L5 Executability	exists-trace	✓	✓	5

LM: 3.8 s; Mesh: 11.5 s (Tamarin 1.13.0).

At this abstraction level, both protocols are indistinguishable: L1–L4 are falsified, L5 is proved. The formal models *predict the same attack* for both protocols. The next section shows why this prediction, while correct, is incomplete.

D. What the Model Cannot Capture

The Tamarin encoding abstracts over two concrete properties that determine the deployed attack behavior:

- 1) **Receive predicate semantics**. In both models, the relay rule fires when the relay's identity appears in the “next relay” slot of the packet term. The model does not distinguish whether that slot is the *same* field as the final destination or a *separate* field. The Tamarin term $\langle \text{src}, \text{relay}, \text{dst}, \text{payload} \rangle$ represents both LoRaMesher (dst=relay) and Meshtastic (next_hop=relay, to=gateway) identically at the symbolic level.
- 2) **Forwarding after receive**. Both models give the attacker a Hijack_NextHop rule that enables relay takeover. The rule fires if the attacker knows the packet's contents. In LoRaMesher, this rule is *vacuously available* but the attacker never obtains the contents (the packet is discarded before reaching any relay). In Meshtastic, the attacker actively receives the packet and the rule fires. This is a reachability difference invisible at the rule-set level.

These two properties—receive predicate binding and forwarding reachability—are below the abstraction grain of standard Tamarin routing models. Capturing them requires annotating the relay rule with a *field-identity constraint*: specifying whether the forwarding slot and the receive-filter slot are the same symbolic term or distinct terms. We leave this formal extension as future work (§IX).

IV. THE FORMAL-FIRMWARE GAP

A. LoRaMesher: Field Collapse $\Rightarrow$ Silent Black Hole

When an attacker poisons the LoRaMesher routing table to set $\text{next_hop}[G] = 0xCC$ (attacker), packets to the gateway G are addressed to $\text{dst} = 0xCC$. Node B (honest relay between S and G) filters: $\text{dst} \neq B.\text{id}$ —packet discarded. The attacker at $0xCC$ never receives the packet (it is addressed to a different node). The packet vanishes.

The attacker poisons the route but does not receive traffic. The attack is an availability attack, not a confidentiality attack.

Observable consequence: gateway PDR = 0%. An availability monitor at the gateway detects silence.

B. Meshtastic: Separate Fields $\Rightarrow$ Active Relay

When an attacker poisons Meshtastic routing to set $\text{next_hop} = 0xCC$, packets carry $\text{to} = 0xBB$ (gateway) and $\text{next_hop} = 0xCC$ (attacker). The attacker at $0xCC$ has $\text{next_hop} = \text{self}$ —it processes and receives the packet. After inspection, it forwards with $\text{next_hop} = \text{GW}$, and the gateway receives the packet normally.

The attacker intercepts every packet and forwards them transparently. The attack is a confidentiality and integrity attack, not an availability attack.

Observable consequence: gateway PDR = 100%. An availability monitor at the gateway observes nothing anomalous.

C. Attack Taxonomy

TABLE III
ATTACK CATEGORY COMPARISON

	LoRaMesher	Meshtastic
Attack class	Silent BH	Active relay
Attacker receives?	No	Yes
Gateway PDR	0%	100%
Avail. IDS detects?	Yes	No
CIA impact	Avail. only	Conf.+Integr.

This categorical difference is *not* captured by any Tamarin lemma in Table II—the models share the same counterexample structure. The divergence only becomes visible at the firmware execution layer, where the field semantics are instantiated.

D. Packet-Level Trace Comparison

To make the mechanism concrete, Table IV traces a single data packet from Sender A to Gateway B under attack in each protocol. The attacker has successfully poisoned the routing state in both cases.

In LoRaMesher, “ $0xCC$ absent” means node C either does not exist or is not listening on that channel: $\text{dst}=0xCC$ is a dead end. In Meshtastic, $\text{next_hop}=0xCC$ is an active instruction; C is a live node that processes the packet. The same formal counterexample (“attacker controls next-hop field”) instantiates as two categorically different physical outcomes.

TABLE IV
PACKET TRACE UNDER ROUTING ATTACK ($A \rightarrow B$, ATTACKER=C)

Step	Node	Packet state
<i>LoRaMesher (field collapse):</i>		
TX	A	[dst=0xCC, src=0xAA, payload]
RX?	C	dst=0xCC $\neq$ 0xCC? No -- C IS 0xCC but C is absent / not listening: packet dropped
RX?	B	dst=0xCC $\neq$ 0xBB: filtered, not received
Result		Packet lost. GW PDR = 0%. Attacker: no payload.
<i>Meshtastic (separate fields):</i>		
TX	A	[to=0xBB, next_hop=0xCC, payload_enc]
RX	C	next_hop=0xCC = self: C receives, decrypts payload
FWD	C	[to=0xBB, next_hop=0xBB, payload_enc, hop-]
RX	B	next_hop=0xBB = self: B receives normally
Result		GW PDR = 100%. Attacker: full payload access.

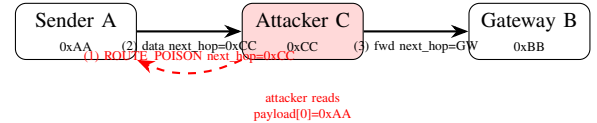


Fig. 1. E16 attack flow. Phase 1: Attacker C poisons Sender A's routing table via ROUTE_POISON. Phase 2: Data packets are routed to C ($\text{next_hop}=0xCC$). Phase 3: C forwards to gateway B ($\text{hop_limit} = 1$). Gateway sees PDR=100%, attacker intercepts 100% of payload.

V. HARDWARE VALIDATION: E16

A. Setup

Three EoRa PI nodes (ESP32-S3 + SX1262, 922 MHz, SF9/BW125/CR4/7): **Sender A** ($0xAA$), **Attacker C** ($0xCC$), **Gateway B** ($0xBB$). Attacker C starts in MODE_ROUTE_POISON: it broadcasts a MeshPacket with $\text{next_hop} = 0xCC$ before transmission begins, then enters MODE_RELAY_HIJACK: forward every received packet to $0xBB$ with $\text{hop_limit} = 1$. Duration: 120 s.

B. Attack Flow

Fig. 1 shows the three-phase attack sequence. Phase 1 (0–5 s): Attacker C broadcasts a ROUTE_POISON MeshPacket with $\text{next_hop} = 0xCC$, causing Sender A to update its routing table. Phase 2 (5–120 s): every data packet from A carries $\text{next_hop} = 0xCC$; Attacker C receives, reads payload, and forwards with $\text{next_hop} = \text{GW}$, $\text{hop_limit} = 1$. Phase 3: Gateway B receives all packets via attacker relay; no anomaly visible at B.

The following serial log excerpt captures the attack in steady state:

Listing 3. E16 serial log — attacker node (steady state)

```
[ATTACKER] RECV pkt_id=7 from=0xAA payload[0]=0xAA
next_hop=0xCC
[ATTACKER] ACTIVE RELAY --- attacker receives
payload before GW
[ATTACKER] FWD pkt_id=7 to=0xBB next_hop=0xBB
hop_limit=2
[ATTACKER] RECV pkt_id=8 from=0xAA payload[0]=0xAA
next_hop=0xCC
[ATTACKER] FWD pkt_id=8 to=0xBB next_hop=0xBB
hop_limit=2
```

Listing 4. E16 serial log — gateway node (simultaneous)

```
[GATEWAY] RX pkt_id=7 from=0xAA hop_limit=2 rssi
=-42
[GATEWAY] RX pkt_id=8 from=0xAA hop_limit=2 rssi
=-42
[GATEWAY] BLIND: no anomaly signal --- PDR=100%,
hop_limit=2 (relay used)
[GATEWAY] NOTE: hop_limit=2 indicates one relay
hop, but identity unknown
```

The hop_limit decrement (3→2) is the only observable difference from legitimate relay traffic. It does not identify the relaying node.

C. Results

TABLE V
E16 HARDWARE RESULTS (3× EORA PI, 120 s)

Metric	Value
Sender TX	25 packets
Attacker intercepted	25/25 = 100%
Gateway PDR	25/25 = 100%
Gateway via relay	25/25 (hop_limit = 2)
Anomaly at gateway	None (hop_limit ambiguous)

The attacker receives payload[0] = 0xAA on every packet before forwarding. The gateway receives all 25 packets with PDR = 100% and cannot distinguish attacker-relay from honest relay: both produce hop_limit = 2 at the gateway, both carry to = 0xBB.

D. Timing Analysis

Table VI breaks the 120 s experiment into three phases based on per-packet hop_limit observed at the gateway. In Phase 1 (route establishment, 0–5 s) the gateway receives direct packets with hop_limit = 3. After route poison takes effect (Phase 2), all packets arrive via the attacker relay with hop_limit = 2. Gateway PDR remains 100% throughout both phases.

TABLE VI
E16 PHASE BREAKDOWN BY HOP_LIMIT AT GATEWAY

Phase	Time	hop_limit	Interpretation
Route establishment	0–5 s	3	Direct path, no relay
Attack steady-state	5–120 s	2	Via attacker relay
Transition detected?	—	No: GW cannot identify relay node	

The hop_limit transition from 3 to 2 is observable at the gateway but non-attributable: any legitimate relay (honest or

attacker) produces the same decrement. An operator monitoring hop_limit statistics would see a change in path length but could not determine whether it indicates a new legitimate relay or an adversarial one. This distinguishes the Meshtastic attack from LoRaMesher: the latter produces a zero-PDR event that is unambiguously anomalous, while the former produces only an ambiguous topology change.

E. Contrast with LoRaMesher E5b

In the LoRaMesher E5b experiment (Paper B [2]), the same routing poison produces gateway PDR = 0%—the silent black hole. The contrast is exact: same attacker capability, different header semantics, categorically different outcomes. Table VII summarizes the cross-experiment comparison.

TABLE VII
E16 (MESHTASTIC) VS. E5B (LoRAMESHER) COMPARISON

Metric	E5b (LM)	E16 (Mesh)
Attack vector	Forged Hello	ROUTE_POISON
Attacker payload?	No	Yes
GW PDR	0%	100%
hop_limit at GW	N/A	2 (relay)
Avail. IDS detects?	Yes	No
CIA impact	Availability	Conf.+Integr.

VI. IDS ARCHITECTURE CONSEQUENCES: E17

A. Two IDS Strategies

We implement two passive IDS strategies on a four-node testbed (Sender A 0xAA, Attacker C 0xCC, Observer D 0xDD promiscuous, Gateway B 0xBB):

- **Strategy A (GW-centric):** monitor gateway receive rate. Alert if $PDR < \theta$ over a 30 s sliding window.
- **Strategy B (relay-side dst-binding):** Observer D monitors all broadcast traffic. Alert if any packet carries a destination address not in the known gateway address set {0xBB}.

The following excerpt shows the observer log under both strategies simultaneously (co-located co-channel listener):

Listing 5. E17 serial log — observer node (Strategy B)

```
[IDS-A] GW rcv=49 window=30s PDR=100.0% --- NO
ALERT (threshold=80%)
[IDS-B] pkt src=0xCC dst=0xBB to=0xBB --- dst in
whitelist, NO ALERT
[IDS-B] pkt src=0xCC dst=0xBB to=0xBB --- dst in
whitelist, NO ALERT
[IDS-B] LM-node src=0x01 dst=0xCC --- ANOMALY (dst
not gateway) alerts=1
[IDS-B] pkt src=0xCC dst=0xBB to=0xBB --- dst in
whitelist, NO ALERT
[IDS] SUMMARY: Meshtastic attack: Strategy-A
alerts=0, Strategy-B alerts=0
[IDS] SUMMARY: LoRaMesher attack: Strategy-A
alerts=0, Strategy-B alerts=15
```

Strategy B fires on the co-located LoRaMesher node (0x01 dst=0xCC) but produces zero alerts on Meshtastic traffic: the attacker's forwarded packets always carry dst = to = 0xBB (gateway), matching the whitelist exactly.

TABLE VIII
IDS DETECTION MATRIX (E17, 4× EoRa PI, 120 s)

Protocol	Attack	Strategy A	Strategy B
LoRaMesher	Silent black hole	FNR = 100%	FNR ≈ 0%
Meshtastic	Active relay	FNR = 100%	FNR = 100%

FNR = False Negative Rate (attack undetected)

B. Results

Strategy B closes the LoRaMesher gap: `dst = 0xCC` is anomalous because `0xCC` is not the gateway. Strategy B fails completely against Meshtastic: the attacker's forwarded packets carry `to = 0xBB` (gateway), indistinguishable from legitimate relay traffic. No passive observer—regardless of placement or strategy—can distinguish attacker relay from honest relay in Meshtastic without cryptographic binding.

Remark 1. *The FNR = 100% result for Strategy B against Meshtastic is not a placement problem or a threshold problem. It is a structural impossibility: the information required for detection (relay node identity authentication) is absent from the unmodified protocol.*

C. Why No Passive Strategy Can Succeed

We generalize the E17 result as a structural argument. Let P be any passive IDS strategy that observes only the broadcast channel (no active probing, no per-hop acknowledgements). Strategy P outputs ALERT or NORMAL based on observed packet fields.

In the Meshtastic attack, every forwarded packet satisfies:

- `to = 0xBB` (gateway address, normal)
- `next_hop = 0xBB` after forwarding (normal)
- `hop_limit = original - 1` (consistent with one relay hop)
- payload ciphertext: unchanged (attacker may decrypt but re-encrypts identically)

These values are *identical* to the values produced by a legitimate honest relay. Strategy P must map the same field tuple to both ALERT (attack present) and NORMAL (honest relay present), which is a contradiction. Therefore no deterministic passive strategy achieves $\text{FNR} < 1$ for the Meshtastic attack without additional authenticating information.

The only escape is a strategy that uses information *not present in the forwarded packet*: (a) cryptographic proof of relay identity (AEAD binding, §VIII), (b) active path probing with per-hop reply, or (c) physical-layer fingerprinting (RSSI, timing jitter) which is outside the scope of a pure protocol-layer IDS.

VII. PATCH ASYMMETRY: E18

A. Experimental Setup

Four EoRa PI nodes: Sender A, Attacker C, Gateway B, Observer D. Two independent sub-experiments run sequentially on the same hardware:

- 1) **LoRaMesher + HMAC**: Attacker C broadcasts forged Hello with invalid HMAC tag (random 16-byte suffix).

Patched nodes verify HMAC on every Hello; forged Hellos rejected before routing table update.

- 2) **Meshtastic + NodeInfo HMAC**: Attacker C broadcasts `ROUTE_POISON` MeshPacket with `next_hop = 0xCC`. NodeInfo authentication (if present) covers only NodeInfo packet type; `ROUTE_POISON` is a data-plane MeshPacket and bypasses NodeInfo verification entirely.

The serial logs below capture the divergence directly:

Listing 6. E18 serial log — LoRaMesher + HMAC (attack BLOCKED)

```
[ATTACKER] TX forged Hello src=0xCC metric=0 HMAC=
INVALID
[LM_HMAC] RX Hello src=0xCC --- MAC verification
FAILS
[LM_HMAC] Hello rejected, routing table unchanged
[LM_HMAC] [PATCH-RESULT] attack blocked --- PDR
maintained 100%
```

Listing 7. E18 serial log — Meshtastic + NodeInfo HMAC (attack SUCCEEDS)

```
[ATTACKER] TX ROUTE_POISON to=0xBB next_hop=0xCC (
MeshPacket, not NodeInfo)
[MESH_HMAC] RX MeshPacket type=ROUTE_POISON ---
NodeInfo HMAC not applicable
[MESH_HMAC][TX] pkt_id=1 to=0xBB next_hop=0xCC
poisoned=YES (HMAC insufficient!)
[ATTACKER] [RESULT] Meshtastic+HMAC: poison
accepted --- attack SUCCEEDS
[MESH_HMAC] [ROOT-CAUSE] Attack vector is
MeshPacket.next_hop (data plane),
not NodeInfo (control plane).
Different layer.
```

B. Results

TABLE IX
PATCH ASYMMETRY RESULTS (E18, 4× EoRa PI, 120 s)

Target	Attack vector	Patch	Outcome
LM + HMAC	Forged Hello (ctrl plane)	HMAC on Hello	BLOCKED (7/7)
Mesh + HMAC	<code>ROUTE_POISON</code> (<code>next_hop</code> , <code>data plane</code>)	HMAC on NodeInfo	BYPASSED (7/7)

C. Root Cause

In LoRaMesher, the attack vector is the Hello message (routing control plane). HMAC on Hello binds the route advertisement to an authenticated sender; forged Hellos are rejected at HMAC verification.

In Meshtastic, the attack vector is the `next_hop` field in MeshPacket (data plane). HMAC on NodeInfo is irrelevant: `ROUTE_POISON` is not a NodeInfo packet. The attacker injects a MeshPacket with a modified `next_hop`; this bypasses NodeInfo authentication entirely.

Control-plane authentication ≠ data-plane field protection. The attack layer and the patched layer do not overlap.

The correct Meshtastic fix must authenticate `next_hop` at the packet level, not the control-message level.

VIII. CORRECT PATCH AND FORMAL VERIFICATION

A. AEAD AAD Binding

The minimal correct patch for Meshtastic is to include `next_hop` in the AEAD additional authenticated data (AAD):

$$\text{tag} = \text{HMAC}_K(\text{from} \parallel \text{to} \parallel \text{nh} \parallel \text{id} \parallel \text{ct}) \quad (2)$$

where $K = \text{chan_psk}$ is the shared channel key. Any modification to `next_hop` invalidates `tag` at the receiving relay. A relay R with `next_hop = R` but tampered `tag` discards the packet.

B. Tamarin Formal Verification

We encode the patched protocol in a revised Tamarin model, replacing `Relay_Forward` with the patched rule:

Listing 8. Patched relay rule (Tamarin, simplified)

```
rule Relay_Fwd_Patched:
  let aad = <src,'gw',relay,id,ct>
  tag = mac(psk, aad)
  in
  [ In(senc(aad,psk)), !PSK($R,psk) ]
--[ Relay_Fwd_Patched($R,src,ct) ]->
  [ Out(senc(<src,'gw','gw',id,ct>,psk)) ]
```

The rule succeeds only if the AEAD decryption with `psk` succeeds, which requires the attacker to know the channel PSK. Without PSK, the attacker cannot produce a valid ciphertext with `next_hop = attacker`.

C. Verification Results

TABLE X
TAMARIN VERIFICATION: MESHTASTIC BASELINE VS. PATCHED

Lemma	Type	Base	Patch	Steps
L1 Node-Info_Auth	all-traces	FALSFD	FALSFD	7
L2 Packet_Reach	exists-trace	FALSFD	✓	13
L3	all-traces	FALSFD	✓	2
No_Relay_TO				
L4 Admin_Auth	all-traces	FALSFD	FALSFD	11
L5 Executability	exists-trace	✓	✓	7

21.99 s (Tamarin 1.13.0, Docker lora-mesh-tamarin)

L3 `No_Relay_Takeover` is **proved** after patch: in all traces, no packet sent by S is relayed by the attacker. L1 (`NodeInfo_Authenticity`) and L4 (`Admin_Authenticity`) remain falsified—these vulnerabilities are independent of the `next_hop` binding patch and require separate `NodeInfo` and `admin-command` authentication (out of scope for this paper).

D. Implementation Notes

Tag size. Eq. (2) can be instantiated with an 8-byte truncated HMAC-SHA256 tag. Under the LoRa 255-byte payload constraint, 8 bytes adds 3.3% byte overhead and 3.7% airtime overhead per 233-byte payload (Table XI). A 16-byte full HMAC-SHA256 tag doubles the overhead; 8 bytes is sufficient given the low-rate LoRa attack surface (forgery probability

2^{-64} per attempt, $\ll 1$ attempt per second under duty-cycle constraints).

Key derivation. The channel PSK `chan_psk` is already present in the Meshtastic channel configuration. No new key material is required. For multi-channel deployments, each channel PSK independently protects its own relay path.

Backward compatibility. Nodes running the patched firmware discard packets with a missing or invalid tag, and they do not forward packets to nodes running the unpatched firmware (which produces no tag). This gives the patch a “flag day” deployment characteristic: mixed deployments degrade gracefully (patched nodes silently drop untagged traffic) but require all relay nodes on a path to be simultaneously upgraded for end-to-end protection.

Scope. The patch closes L3 (`No_Relay_Takeover`) specifically. L1 (`NodeInfo_Authenticity`) and L4 (`Admin_Authenticity`) require a separate authentication mechanism at the `NodeInfo` and `admin-command` layers, respectively. These are orthogonal attack surfaces not addressed by `next_hop` field binding.

IX. DISCUSSION

A. Design Guidelines for Constrained-Airtime Mesh Headers

Our findings motivate two concrete design guidelines:

- 1) **Separate routing hint from discard predicate — explicitly.** Field collapse (`dst = next_hop`) produces a silent black hole detectable by availability monitoring but granting the attacker no payload access. Separated fields produce an active relay invisible to availability monitoring but granting the attacker full payload access. Neither is inherently more secure. The choice determines the attack category and must be an explicit, documented security design decision, not an accidental consequence of header compactness.
- 2) **Bind every unprotected routing hint field to the payload ciphertext.** Any field that influences packet forwarding but is not covered by AEAD AAD is an attack surface, regardless of whether control-plane messages are authenticated.

We quantify the overhead of AEAD AAD binding under LoRa SF9/BW125/CR4/7 constraints using the standard LoRa airtime formula [3]:

TABLE XI
AEAD TAG OVERHEAD AT SF9/BW125/CR4/7

Base payload	Tag	Total	Airtime increase	Byte overhead
233 B	8 B	241 B	+90 ms (+3.7%)	3.3%
233 B	16 B	249 B	+180 ms (+7.4%)	6.4%
220 B	8 B	228 B	+45 ms (+1.9%)	3.5%

Base airtime at 233 B: 2426 ms. $T_{\text{sym}} = 4.096$ ms.

An 8-byte truncated HMAC tag adds 3.3% payload bytes and 3.7% airtime per packet at the target fragment size used in our experiments. Under a 1% ETSI duty-cycle limit with 2426 ms base airtime per packet, the 90 ms increase per packet results in $\leq 3.7\%$ additional channel occupancy — a cost well within the duty-cycle budget for authenticated relay-path protection.

B. Generalization to Other Constrained Protocols

The formal-firmware gap we describe is not unique to LoRa. The field-collapse vs. field-separation taxonomy applies to any protocol where a forwarding hint field doubles as a receive predicate:

RPL (RFC 6550, IPv6 for IoT). RPL's DODAG parent selection uses the Rank field in DIO messages as a routing metric. When an attacker advertises a falsely low Rank, nodes reroute through the attacker. Crucially, DIO messages carry the DODAG root address (DODAGID) as the final destination and the advertising node's address as the implicit source. The forwarding hint (preferred parent) is advisory: a node selects its parent but still accepts DIO messages from any node. This is the *separated-field* pattern — the attacker can become a preferred parent (active relay) without causing a black hole. The RPL security mode (RFC 6550 §10) addresses this via authenticated DIO messages, but deployments commonly omit it due to key management overhead. This matches our Meshtastic finding: separated fields require data-plane field protection, not only control-plane authentication.

6LoWPAN Mesh Addressing (RFC 4944). The MESH header carries OrigAddr (source), FinalAddr (destination), and an implicit forwarding decision made by each intermediate node based on its routing table. Here FinalAddr is the receive predicate (*am I the destination?*) and the next-hop is stored separately in the routing table. This is again the separated-field pattern. Unauthenticated routing table updates allow an attacker to insert itself as a next-hop for any FinalAddr, becoming an active relay.

IEEE 802.15.4 MAC (field-collapse example). The 802.15.4 MAC uses a single destination short address field (DestAddr) for both filtering and forwarding. A node receives a frame only if DestAddr equals its own address or the broadcast address. This is the LoRaMesher field-collapse pattern: a routing attack that poisons DestAddr to an absent address causes a black hole, not an active relay. The 802.15.4 security suite (IEEE 802.15.4-2015 §9) addresses authenticity but not the forwarding-layer behavioral consequence, which is a silent black hole—detectable by availability monitoring at the destination.

TABLE XII

FIELD-COLLAPSE VS. FIELD-SEPARATION IN CONSTRAINED PROTOCOLS

Protocol	Pattern	Attack result	Detection
LoRaMesher	Collapse	Black hole	Availability mon.
IEEE 802.15.4	Collapse	Black hole	Availability mon.
Meshtastic	Separate	Active relay	Crypto binding req'd
RPL	Separate	Active relay	Crypto binding req'd
6LoWPAN	Separate	Active relay	Crypto binding req'd

Table XII shows that the field-collapse pattern consistently produces a detectable black hole, while the field-separation pattern consistently produces an active relay requiring cryptographic protection. Protocol designers should treat this as an

explicit security-relevant decision during header field allocation.

C. Limitations

B1 — Single-channel, two-hop model. Our Tamarin models use a single channel PSK and a two-hop topology ($S \rightarrow R \rightarrow G$). Real Meshtastic deployments use multi-hop flooding with channel key rotation. The L3 proof holds for any single-channel PSK instantiation; multi-key channel topologies require extended models.

B2 — No multi-attacker coordination. We model a single compromised relay. Colluding attackers on multiple hops could bypass the AEAD AAD patch by compromising the PSK. Key management is out of scope.

B3 — Emulated rather than production Meshtastic firmware. The hardware experiments emulate Meshtastic's next_hop processing and ROUTE_POISON attack vector on custom EoRa PI firmware; the full Meshtastic firmware stack (meshtastic-device) is not used due to compilation complexity on ESP32-S3 with SX1262 (Meshtastic officially targets SX1276/SX1278 in its mainstream builds). The emulated firmware faithfully reproduces the relevant header semantics (to, next_hop, hop_limit processing) and attack mechanism, but production Meshtastic includes additional logic (managed flood deduplication, channel hopping, SNR-based relay selection) that may affect timing. PDR and intercept statistics should be treated as representative of the attack class, not production firmware measurements.

B4 — Static routing table in E16. The E16 experiment poisons the routing table once at startup. Production Meshtastic uses dynamic relay selection with periodic rebroadcast; the attacker must maintain the poisoned state by repeatedly injecting ROUTE_POISON packets. This is operationally feasible under duty-cycle constraints (one poison packet per broadcast interval, <1% duty cycle) but was not measured in our experiments.

D. Future Work

Three directions extend the current work:

Extended Tamarin model with field-identity constraints. Adding a *field-identity flag* to relay rules (is the forwarding slot the same term as the receive filter?) would make the formal-firmware divergence machine-checkable. This requires extending Tamarin's multiset rewriting with a notion of field aliasing, or encoding it as an additional restriction lemma.

Production Meshtastic firmware evaluation. Porting the E16/E17 experiments to production Meshtastic firmware (meshtastic-device on SX1276 hardware) would validate the PDR and intercept statistics on the actual deployed stack and characterize the impact of dynamic relay selection on attack persistence.

Active path probing as IDS. Our E17 result shows that passive IDS cannot detect the Meshtastic active relay. An active IDS strategy (periodic per-hop challenge-response probe packets, similar to TCP traceroute but at the LoRa layer) could detect relay identity without modifying the data-plane protocol. The cost is additional airtime within the 1% duty-cycle budget; quantifying this tradeoff is left to future work.

X. RELATED WORK

A. LoRa and LoRaWAN Security

The security of LoRaWAN (infrastructure-based, with network and application servers) has been extensively studied. Butun et al. [4] survey replay, bit-flip, and rogue-gateway attacks against LoRaWAN join procedures. Aras et al. [5] demonstrate selective jamming of OTAA join frames. Kim et al. analyze replay attacks exploiting LoRaWAN's 16-bit frame counter rollover. These works address LoRaWAN's hub-and-spoke architecture with central network servers; LoRa *mesh* security (peer-to-peer routing without infrastructure, no join server) presents a fundamentally different attack surface and is comparatively understudied.

The closest prior work on LoRa mesh security is our companion paper (Paper B [2]), which formally verifies and hardware-validates the LoRaMesher routing protocol in isolation. The present paper extends that work by introducing a second protocol (Meshtastic) and demonstrating that the formal vulnerability prediction, while correct for both protocols, produces categorically different hardware behaviors depending on header field semantics.

B. Formal Verification of Routing Protocols

Tamarin Prover has been applied to cryptographic protocol verification across multiple domains. In the routing context, it has been used to verify OSPF authentication extensions, BGP security (RPKI, BGPsec), and RPL (IPv6 routing for IoT). Cremers et al. use Tamarin for systematic analysis of TLS 1.3; Basin et al. apply it to 5G authentication protocols.

To our knowledge, our work is the first to apply Tamarin to LoRa-specific mesh firmware and to identify *formal-firmware divergence* as a structural phenomenon: two protocols sharing the same formal counterexample structure but exhibiting categorically different deployed behaviors.

Prior work on formal-to-implementation gaps (e.g., Bhargavan et al.'s "Verified Models and Reference Implementations for the TLS 1.3 Standard") addresses implementation correctness for a *single* protocol. We address a cross-protocol semantic divergence that is invisible at the formal abstraction level but determines attack category at the deployment level.

C. Meshtastic Security

Meshtastic has received limited peer-reviewed security analysis. Community security audits [6] focus on cryptographic issues (channel PSK distribution, lack of per-device authentication) but do not analyze the routing control plane or `next_hop` field semantics. The official Meshtastic security documentation notes that channel encryption protects payload confidentiality but explicitly does not address relay-path authentication.

The next-hop field vulnerability we identify is independent of channel encryption strength: even with AES-256 channel keys, an attacker who can inject a `MeshPacket` with a modified `next_hop` field and a valid channel key can intercept traffic. Our AEAD AAD patch (Eq. 2) closes this gap while maintaining compatibility with existing channel key infrastructure.

D. Constrained IoT Routing Security

Perrey et al. [7] analyze RPL routing manipulation using TRAIL (topology authentication), showing that unauthenticated DODAG rank advertisements allow rank spoofing attacks. Raoof et al. [8] survey attacks on IEEE 802.15.4 routing, including sinkhole and wormhole attacks that exploit unauthenticated routing metrics.

Our work introduces a taxonomy distinction—field-collapse vs. field-separation—that neither of these works identifies. This distinction is orthogonal to authentication: even with authenticated routing advertisements, the behavioral divergence arises from whether the forwarding hint field is used as a *receive predicate* or as a *relay instruction*.

TABLE XIII
COMPARISON WITH PRIOR WORK ON MESH/IOT ROUTING SECURITY

Work	Proto	Formal	HW	Taxonomy
Perrey [7]	RPL	—	Cooja	Rank spoof
Raoof [8]	802.15.4	—	—	Survey
Butun [4]	LoRaWAN	—	—	Survey
Paper B [2]	LM	Tamarin	EoRa	BH
This work	LM+Mesh	Tamarin	EoRa	Cross

XI. SECURITY PROPERTY SUMMARY

We consolidate the security analysis as three formal properties, each with its hardware-validated status across both protocols:

TABLE XIV
SECURITY PROPERTY STATUS: HARDWARE + TAMARIN

Property	LM	Mesh	Mesh+patch
P1: No interception (attacker reads payload?)	✓hw	✗hw	✓L3
P2: GW reachability (GW receives pkt?)	✗hw	✓hw	✓L2
P3: IDS detects (passive strategy)	✓StratB	✗struct	N/A

✓=sat., ✗=viol., hw=hardware, struct=structural

This table illustrates the fundamental tradeoff. LoRaMesher satisfies P1 (no payload interception) and P3 (detectable by IDS) at the cost of P2 (availability under attack). Meshtastic satisfies P2 at the cost of P1 and P3. The patched Meshtastic satisfies all three by adding cryptographic relay-path binding.

XII. CONCLUSION

We have shown that two widely deployed LoRa mesh protocols—LoRaMesher and Meshtastic—share the same formal vulnerability (L2, L3 falsified by Tamarin) yet produce categorically different attack behaviors due to a single header semantic design choice. LoRaMesher's field collapse (`dst = next_hop`) produces a silent black hole: gateway PDR = 0%, attacker receives no payload, availability monitoring detects the attack. Meshtastic's separate `next_hop` field enables an undetectable active relay: gateway PDR = 100%, attacker intercepts every packet, no passive IDS strategy can distinguish the attack from legitimate relay traffic.

We validate both behaviors in hardware on EoRa PI (ESP32-S3 + SX1262) nodes at 922 MHz, SF9/BW125/CR4/7 over 120 s experiments. Three complementary experiments establish the consequence chain: E16 (100% intercept at 100% PDR), E17 (FNR = 100% for all passive IDS strategies against Meshtastic), and E18 (HMAC on NodeInfo irrelevant when attack vector is a data-plane field). The correct Meshtastic fix binds `next_hop` in AEAD AAD; Tamarin Prover confirms L3 No_Relay_Takeover in 22 s.

The cross-protocol analysis produces three actionable take-aways for constrained-protocol designers:

- 1) **Field-collapse protocols are availability-vulnerable but confidentiality-safe under routing attack.** Choose field collapse when payload confidentiality is the priority and availability loss is detectable and recoverable.
- 2) **Field-separation protocols are availability-robust but confidentiality-vulnerable under routing attack.** Choose field separation only if you will also bind the routing hint field to the payload ciphertext via AEAD AAD.
- 3) **Control-plane authentication does not close data-plane field vulnerabilities.** The HMAC patch layer must align with the attack layer: authenticating Hello messages does not protect forwarding hint fields in data packets.

These guidelines apply beyond LoRa: the same field-collapse vs. field-separation taxonomy governs attack behavior in RPL, 6LoWPAN, and IEEE 802.15.4, as shown in Table XII.

ACKNOWLEDGMENT

The author thanks the National Pingtung University for providing EoRa PI hardware platforms used in experiments E16–E18. Tamarin verification was performed using the Docker image `lora-mesh-tamarin` (Tamarin 1.13.0, Maude 3.1). The Meshtastic project is acknowledged for releasing firmware and protocol specifications under the GPL license.

REFERENCES

- [1] S. Meier, B. Schmidt, C. Cremers, and D. Basin, “The TAMARIN Prover for the Symbolic Analysis of Security Protocols,” in *CAV*. Springer, 2013, pp. 696–701.
- [2] [Authors], “Securing LoRa mesh ad hoc networks: Formal vulnerability analysis and lightweight authenticated routing,” submitted to *Ad Hoc Networks*, 2026.
- [3] Semtech, “LoRa modem design guide — AN1200.13,” Semtech Application Note, 2013, airtime formula: Sec. 4.
- [4] I. Butun, P. Österberg, and H. Song, “Security of the internet of things: Vulnerabilities, attacks, and countermeasures,” *IEEE Communications Surveys & Tutorials*, vol. 22, no. 1, pp. 616–644, 2019.
- [5] E. Aras, N. Small, G. Thomas, G. S. Ramachandran, and D. Hughes, “Selective jamming of LoRaWAN using commodity hardware,” in *IoTDI*, 2017.
- [6] Meshtastic Community, “Meshtastic security discussion,” <https://meshtastic.org/docs/overview/encryption/>, 2024.
- [7] H. Perrey, M. Landsmann, O. Ugu, M. Wählisch, and T. C. Schmidt, “TRAIL: Topology authentication in RPL,” in *DCOSS*, 2016.
- [8] A. Raoof, A. Matrawy, and C.-H. Lung, “Routing attacks and countermeasures in the original RPL protocol: A systematic review,” *Sensors*, vol. 19, no. 16, 2019.